

**«Санкт-Петербургский государственный электротехнический университет
«ЛЭТИ» им. В.И.Ульянова (Ленина)»
(СПбГЭТУ «ЛЭТИ»)**

ОТЧЁТ ПО УЧЕБНОЙ ПРАКТИКЕ

Тема: UI-тестирование

Студенты гр. 4382

Калинина А.В
Смирнов Н.А.
Бурая В.С.
Писаренко А.А.

Руководитель

А.М. Шевелёва

Санкт-Петербург

2026

ЗАДАНИЕ НА УЧЕБНУЮ ПРАКТИКУ

Студент: Бурая В.С.

Группа: 4382

Тема практики: UI-тестирование

Задание на практику:

Проектирование и разработка программного комплекса для автоматизированного UI-тестирования веб-приложения. В рамках работы требуется составить детальный чек-лист, содержащий 10 тестовых сценариев, состоящих суммарно не менее чем из 50 действий, и реализовать их на языке Java с использованием фреймворков Selenide и JUnit 5. Архитектурное решение должно опираться на паттерн Page Object Model и компонентный подход, обеспечивая независимость тестовой логики от структуры страниц. Дополнительно была интегрирована система логирования Log4j2 для отслеживания хода выполнения тестов, ведение совместной работы в репозитории GitHub и оформление отчета с описанием структуры классов и UML-диаграммами.

Сроки прохождения практики: 06.07.2026 – 18.07.2026

Дата сдачи отчета: 18.07.2026

Дата защиты отчета: 18.07.2026

Студенты

Руководитель

Калинина А.В.
Смирнов Н.А.
Бурая В.С.
Писаренко А.А.

А.М. Шевелёва

АННОТАЦИЯ

Данный отчет описывает разработку и программную реализацию масштабируемой системы автоматизированного UI-тестирования для образовательной платформы Stepik.org. В рамках работы был выполнен анализ функциональных возможностей веб-ресурса и разработан подробный чек-лист, содержащий десять ключевых тестовых сценариев. На языке программирования Java спроектирован тестовый фреймворк, основанный на паттерне Page Object Model, что позволило изолировать логику тестов от структуры веб-интерфейса. Практическая реализация выполнена с использованием инструментов Selenide и JUnit 5. В проект интегрирована система логирования Log4j2 для мониторинга выполнения тестов, подготовлена техническая документация с UML-диаграммами классов, а итоговый программный продукт размещен в совместном репозитории GitHub.

SUMMARY

This report describes the development and software implementation of a scalable automated UI testing system for the Stepik.org educational platform. As part of the work, an analysis of the web resource's functionalities was conducted, and a detailed checklist containing ten core test scenarios was developed. An automated testing framework has been designed in the Java programming language based on the Page Object Model pattern, separating test logic from the web interface structure. The practical implementation was carried out using Selenide and JUnit 5 tools. The Log4j2 logging system has been integrated into the project for test execution monitoring, technical documentation with UML class diagrams has been prepared, and the final software suite is hosted in a collaborative GitHub repository.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	6
1 ПОСТАНОВКА ЗАДАЧИ	8
2 РЕЗУЛЬТАТЫ РАБОТЫ.....	23
2.1. Описание программных классов и их методов	23
2.2. UML-диаграмма классов	25
2.3. Реализация взаимодействия компонентов.....	26
3 ИНДИВИДУАЛЬНАЯ РАБОТА	28
4 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ	30
5 ОТЗЫВ РУКОВОДИТЕЛЯ	31
ЗАКЛЮЧЕНИЕ	32
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	34

ВВЕДЕНИЕ

Предметной областью данной практики выступает автоматизация функционального тестирования пользовательских интерфейсов (UI) веб-приложений. Ручная верификация может занимать большое количество времени и часто подвергается влиянию человеческого фактора. Автоматизация тестирования таких элементов интерфейса, как модули глобального поиска, фильтры и блоки навигации, обеспечивает стабильность функционирования важных компонентов программного обеспечения и гарантирует корректность представления данных при изменениях кода.

Целью практики является разработка и программная реализация масштабируемой, объектно-ориентированной архитектуры автоматизированного тестирования. Основной задачей было создание гибкого тестового фреймворка, минимизирующего дублирование программного кода, упрощающего сопровождение автотестов и повышающего надежность проверок за счет современных средств автоматизации и паттернов проектирования.

Для достижения поставленной цели в ходе выполнения работы были решены следующие задачи:

1. Выполнить детальный анализ функциональных возможностей исследуемой веб-платформы и составить чек-лист из десяти сценариев, охватывающих основные пользовательские пути;
2. Разработать программную структуру проекта на языке Java с использованием паттерна Page Object Model [1] и компонентно-ориентированного подхода для изоляции тестовой логики от особенностей реализации интерфейса;
3. Программно реализовать автотесты на базе библиотек Selenide [2] и JUnit 5 [3], внедрить систему логирования Log4j2 [4] для контроля процессов выполнения проверок, а также составить техническую документацию, содержащую визуализацию структуры классов в виде UML-диаграмм.

Также необходимо было эффективно распределить задачи между участниками проекта.

Описание вклада каждого участника:

Калинина А.В.: реализация Page Objects (HomePage, SearchResultsPage, CoursePage, ProfilePages), создание чек-листа, настройка логирования, подготовка и финализация README, приведение классов к единому порядку, вынесение Selenide элементов в переменные, удаление неиспользуемых методов и импортов, добавление документации к классам pages и BaseTest.

Смирнов Н.А.: Архитектура проекта и базовая настройка: создание Maven проекта, подключение зависимостей (Selenide, JUnit5, Log4j2), реализация папки components (SearchComponent, FilterComponent, CourseCardItem и т.д.), настройка BaseTest, инициализация структуры проекта, работа над чек-листом, проверка архитектуры кода и проверка соблюдения принципов Page Object Model, приведение классов к единому порядку, вынесение Selenide элементов в переменные, удаление неиспользуемых методов и импортов.

Бурая В.С.: Тесты №1–5: навигация по страницам, поиск с фильтрами, фильтрация по ценовому диапазону, поиск для продвинутых с сертификатом, сброс текстового поиска "крестиком". Проверка стабильности выполнения тестов. Исправление и добавление методов. Удаление всех комментариев с действиями из тестов. Вынесение тестов с фильтрацией в отдельный класс.

Писаренко А.А.: Тесты №6–10: смена запроса с сохранением фильтра, несуществующий запрос с фильтрами, поиск по автору с переходом в профиль, фильтрация акционных курсов, добавление курса в избранное из поиска. Добавление assert-проверок и обработка исключений. Проверка работы всех тестов в связке. Исправление и добавление некоторых методов. Создание отдельного класса для работы с окнами. Вынесение всех магических чисел в константы.

1 ПОСТАНОВКА ЗАДАЧИ

В итоговом проекте имеются 10 тестов:

1. Текстовый поиск и навигация по страницам
2. Поиск с фильтрами
3. Фильтрация по ценовому диапазону
4. Поиск для продвинутых с сертификатом
5. Сброс текстового поиска «крестиком»
6. Смена запроса с сохранением фильтра
7. Несуществующий запрос с фильтрами
8. Поиск по автору с переходом в профиль
9. Фильтрация акционных курсов
10. Добавление курса в избранное из поиска

Тест №1: Текстовый поиск и навигация по страницам

Данная проверка подтверждает корректность работы поисковой строки при вводе ключевых слов, последовательное переключение страниц выдачи и последующее открытие страницы конкретной учебной программы.

Результаты действий представлены на рисунках (см. рисунки 1 – 4).

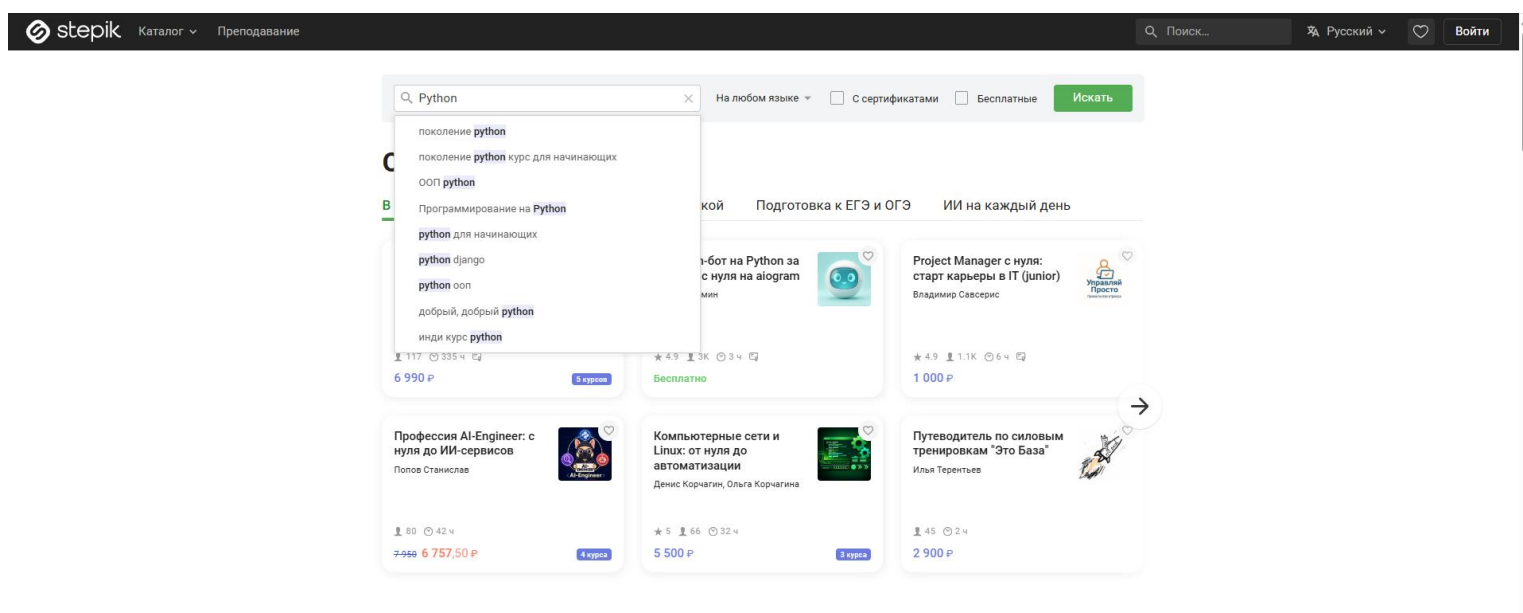


Рисунок 1 – Введение в поиск запрос «Python»

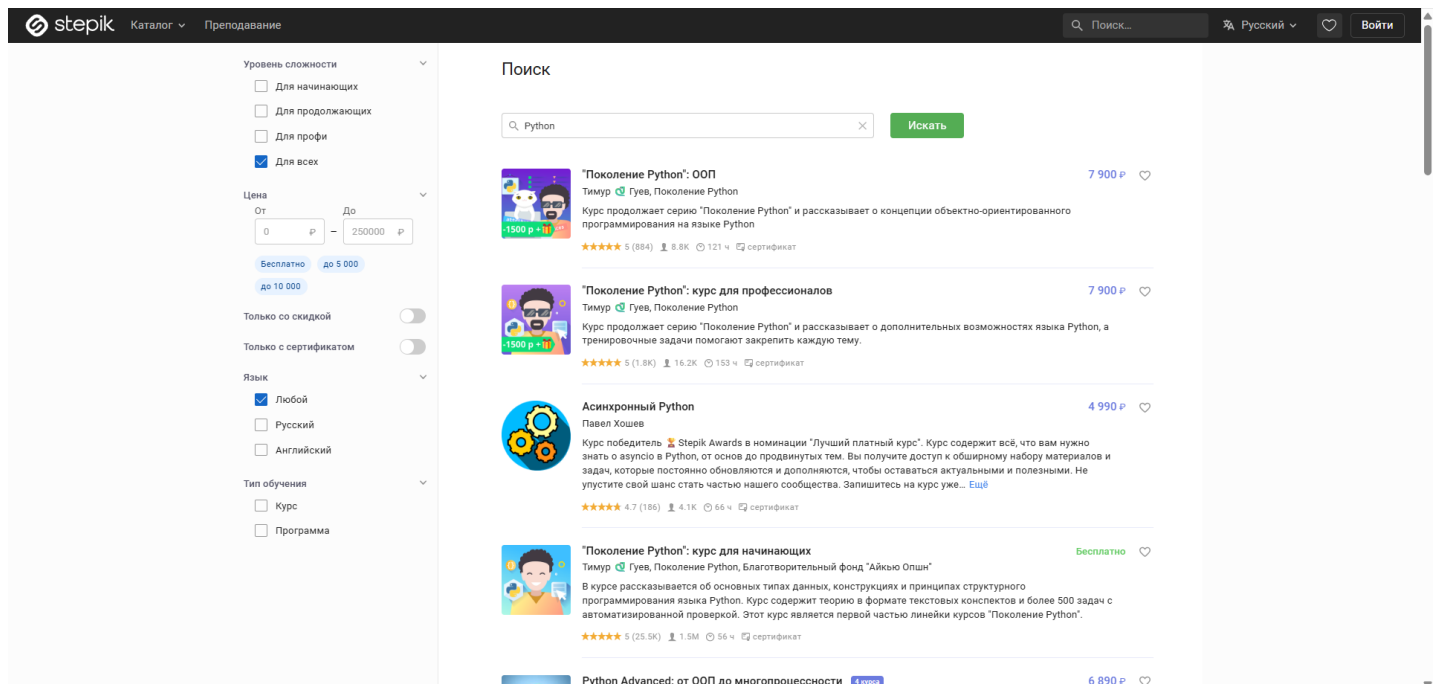


Рисунок 2 – Результаты поиска по запросу «Python»

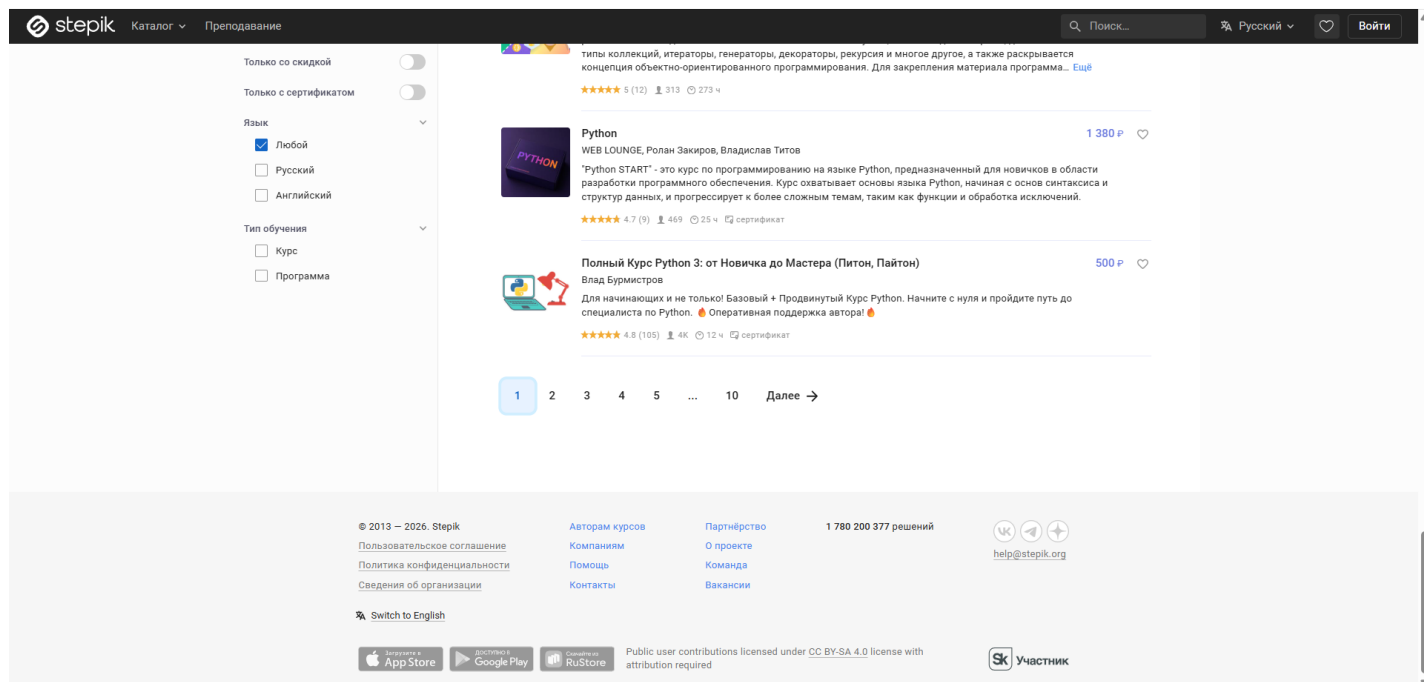


Рисунок 3 – Прокрутка вниз к пагинации

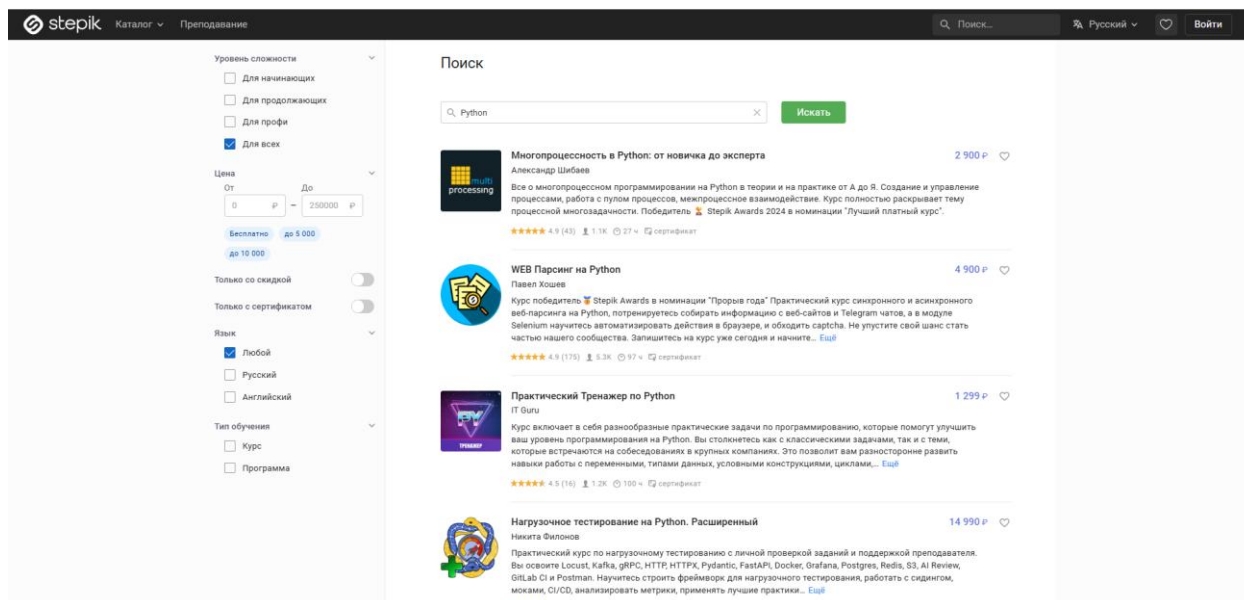


Рисунок 4 – Переход на вторую страницу запросов

Тест №2: Поиск с фильтрами

В рамках теста оценивается корректность отбора результатов поиска при одновременном использовании языковых критериев и фильтров по типам учебных курсов.

Результаты действий представлены на рисунках (см. рисунки 5, 6).

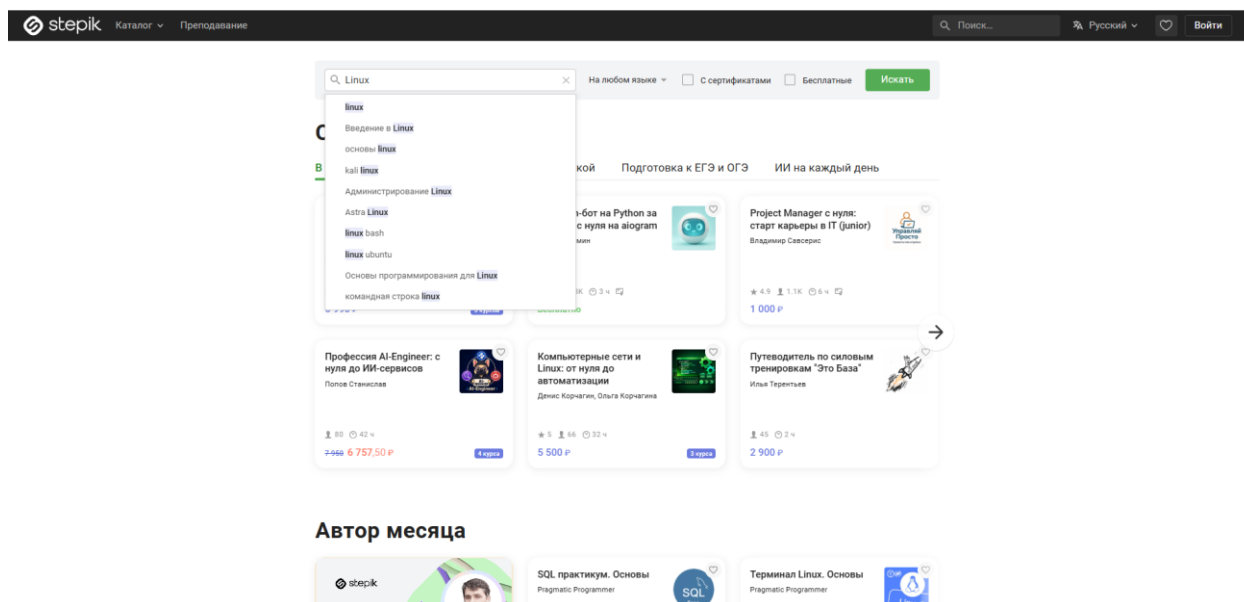


Рисунок 5 – Введение в поиск запрос «Linux»

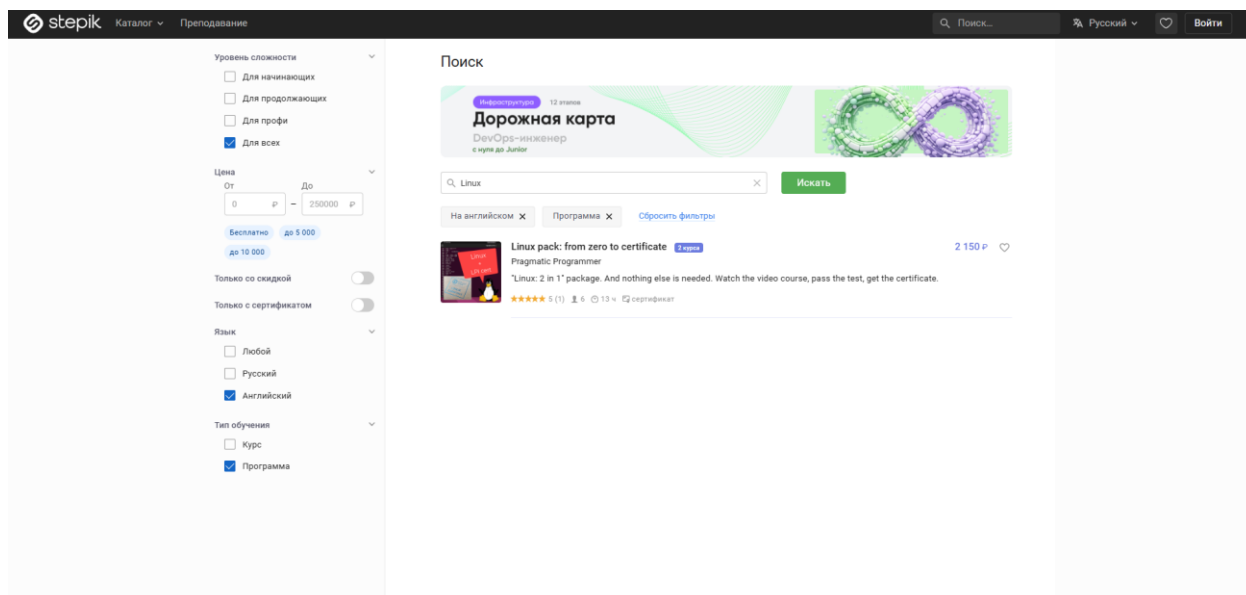


Рисунок 6 – Применение чекбоксов «Английский» и «Программа»

Тест №3: Фильтрация по ценовому диапазону

В рамках теста контролируется корректность ограничения поисковой выдачи по стоимости с помощью ввода значений в поля минимального и максимального ценовых порогов.

Результаты действий представлены на рисунках (см. рисунки 7, 8).

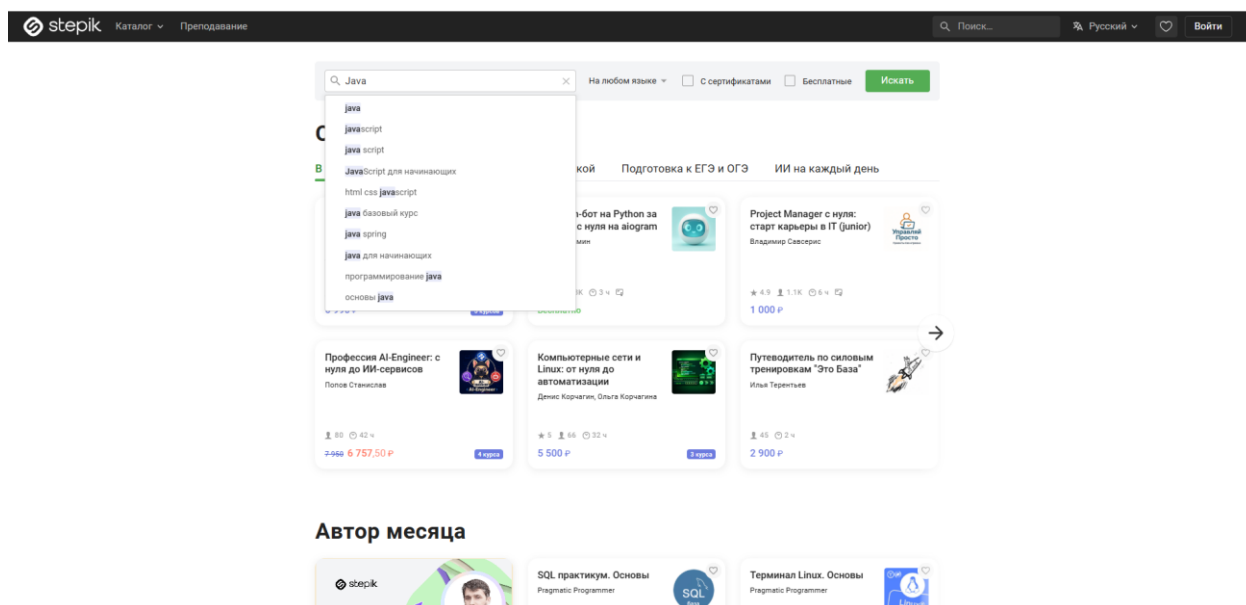


Рисунок 7 – Введение в поиск запрос «Java»

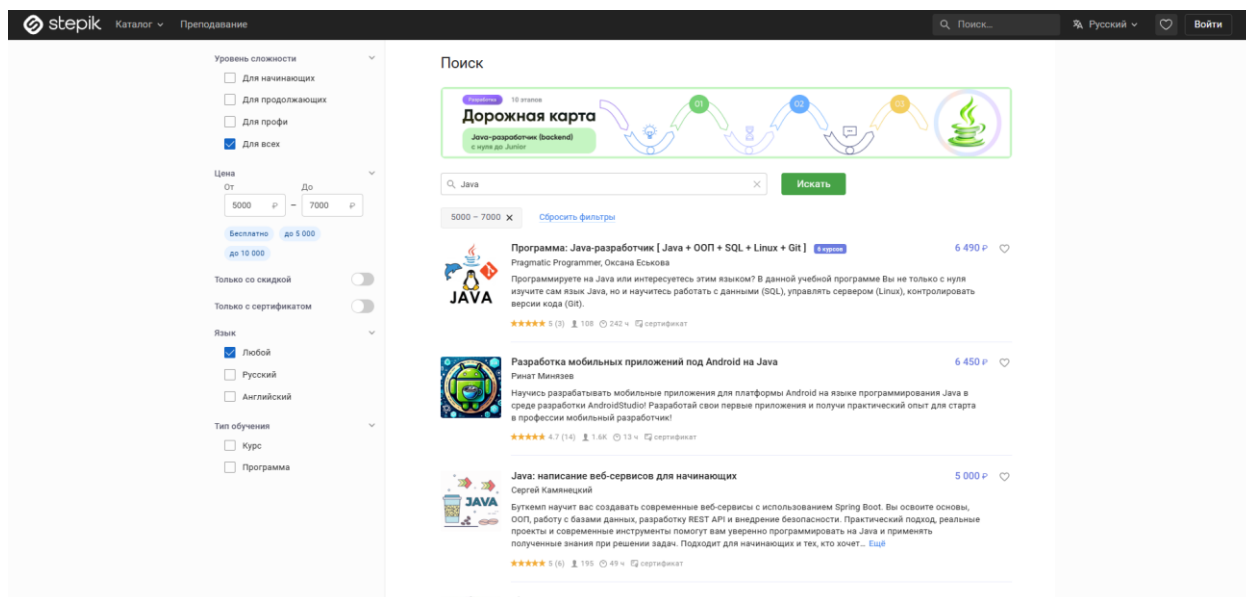


Рисунок 8 – Применение фильтра по цене

Тест №4: Поиск для продвинутых с сертификатом

Тест контролирует корректность поисковой выдачи при комбинации критерия «Продвинутый уровень» и требования о наличии выдаваемого сертификата по окончании программы.

Результаты действий представлены на рисунках (см. рисунки 9 – 10).

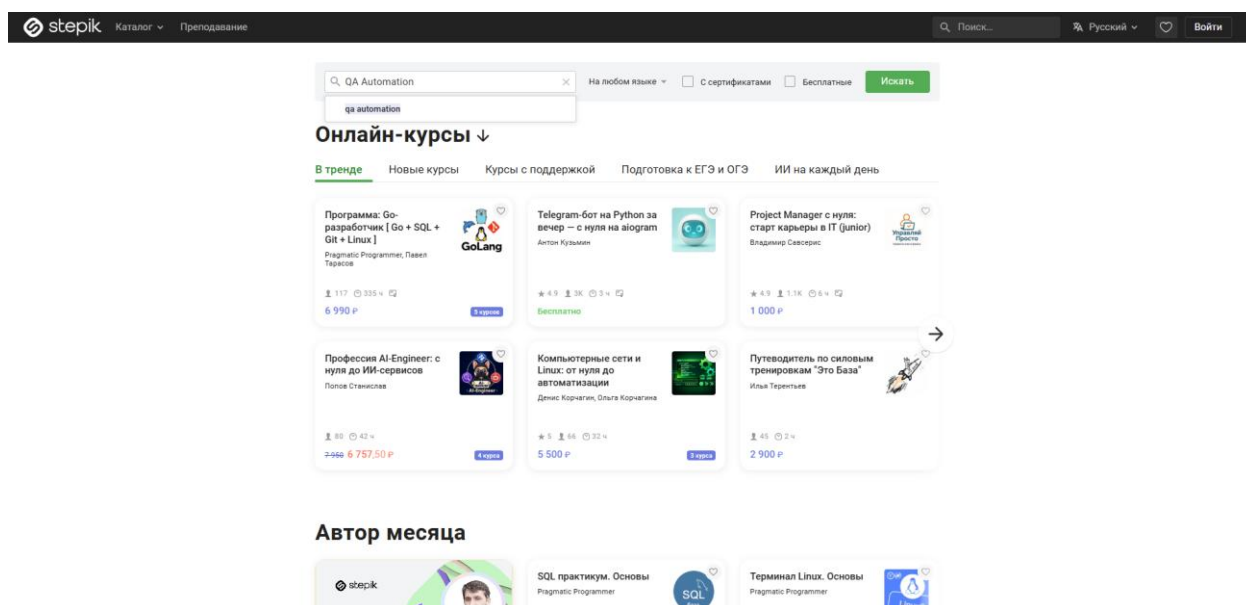


Рисунок 9 – Введение в поиск запрос «QA Automation»

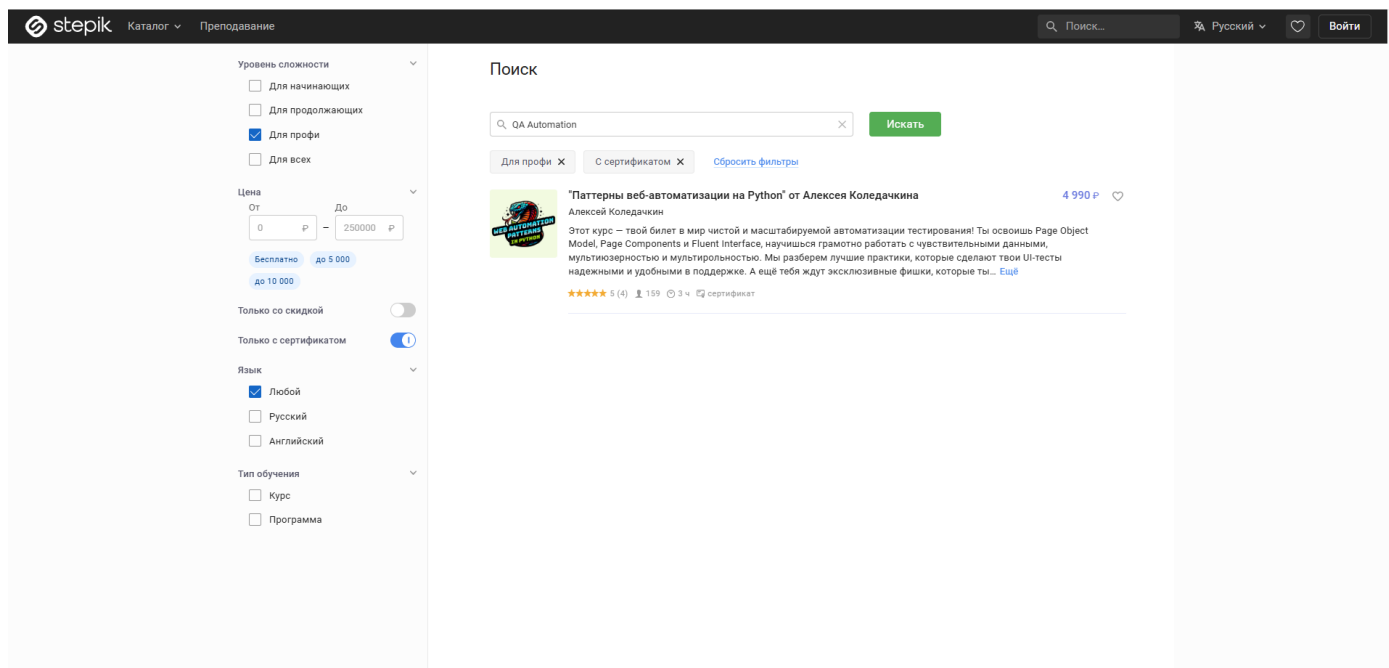


Рисунок 10 – Применение чекбоксов «Для профи» и «Только с сертификатом»

Тест №5: Сброс текстового поиска «крестиком»

Данная проверка контролирует процесс очистки поискового запроса кликом по «крестику», проверяя стирание текста в поле поиска и автоматическое возвращение каталога к его дефолтному отображению.

Результаты действий представлены на рисунках (см. рисунки 11 - 13).

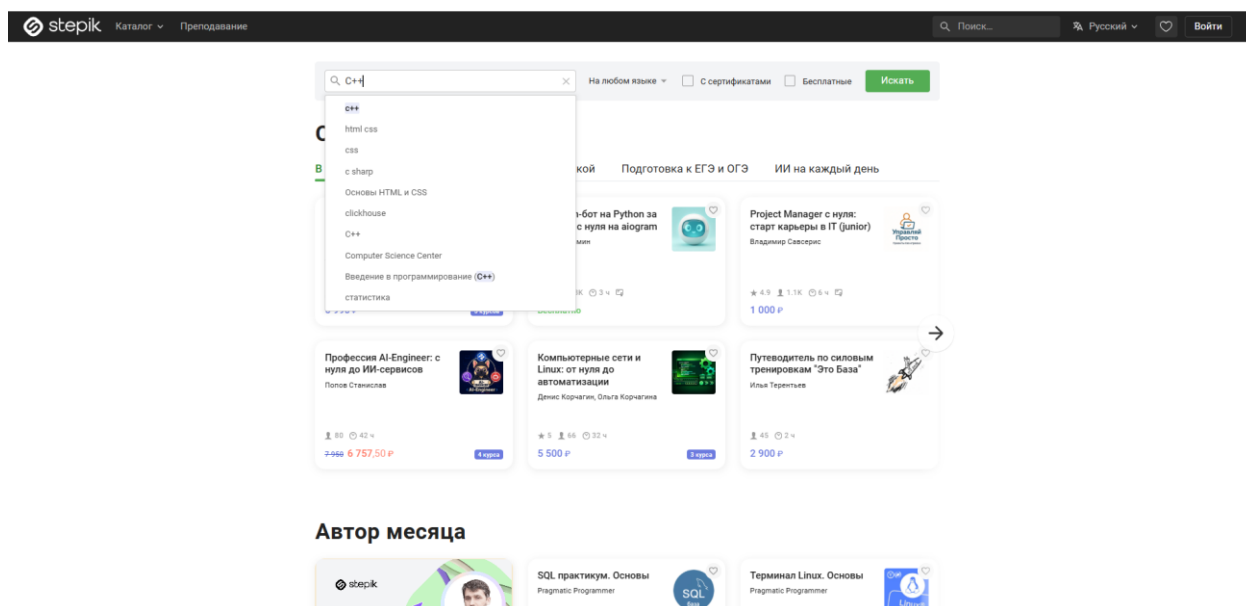


Рисунок 11 – Введение в поиск запрос «C++»

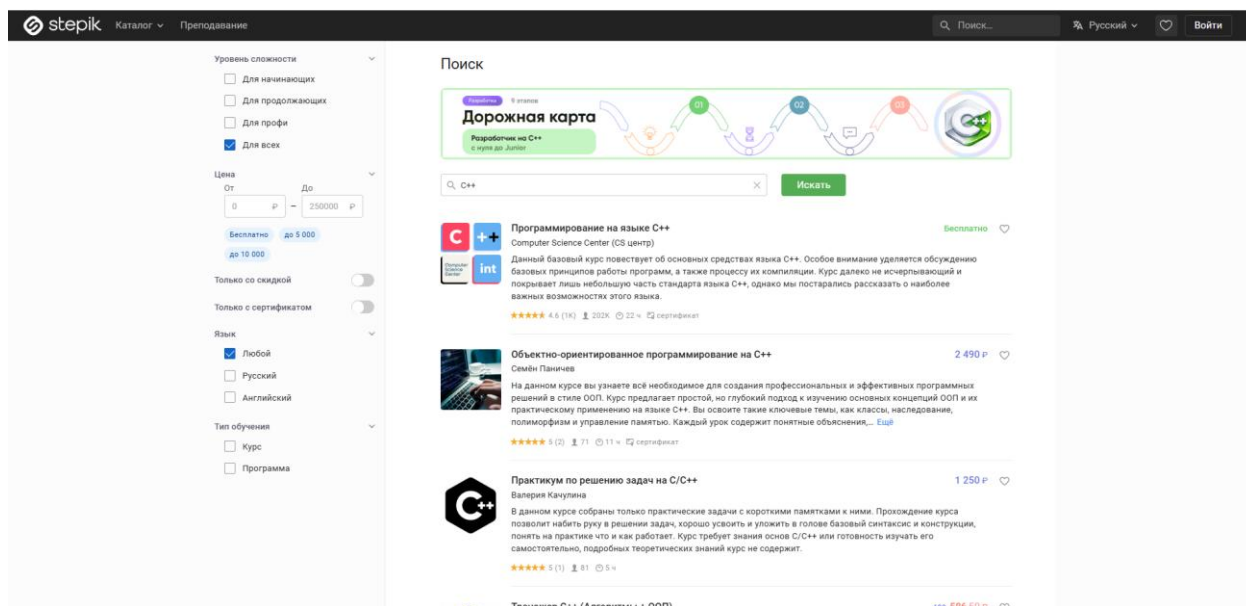


Рисунок 12 – Вывод всех доступных курсов

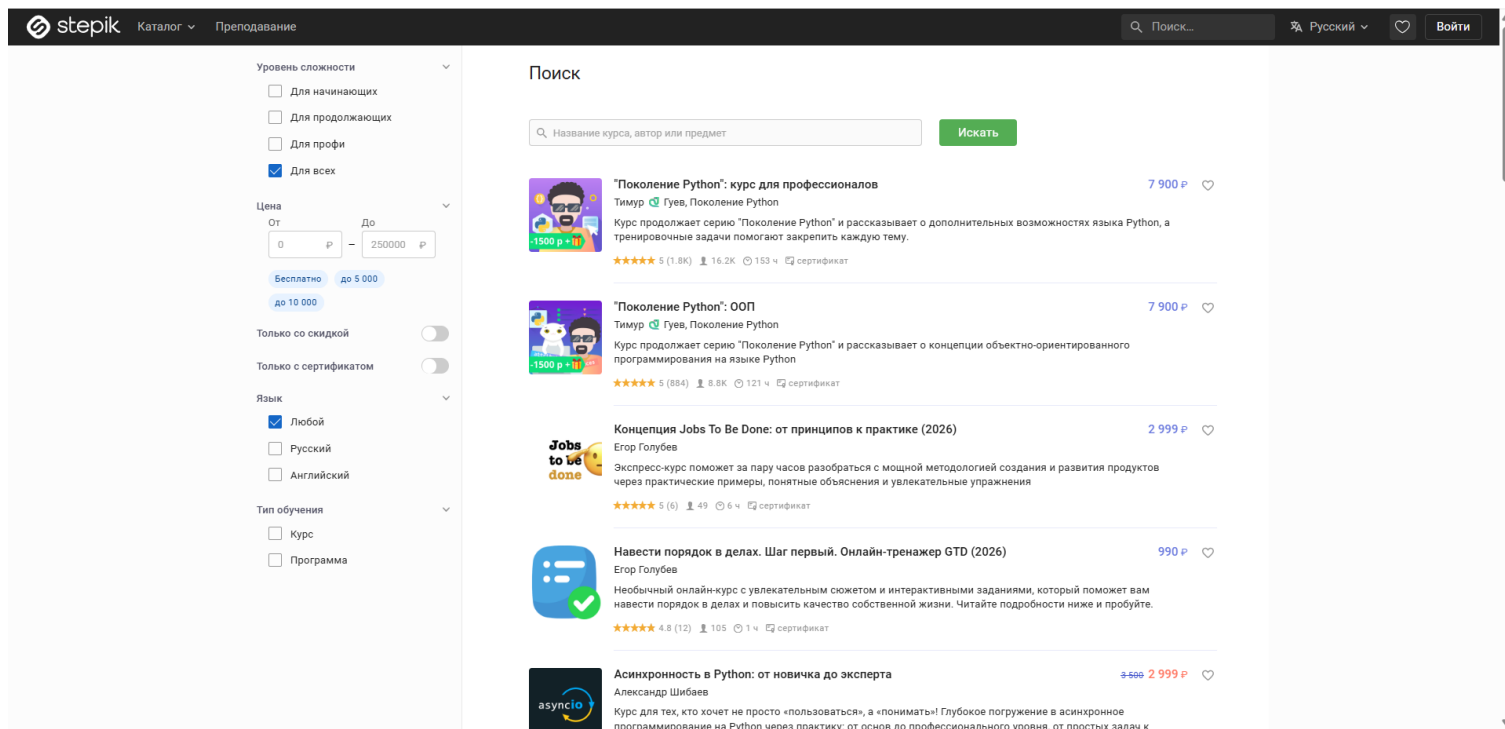


Рисунок 13 – Нажатие крестика

Тест №6: Смена запроса с сохранением фильтра

Тест контролирует корректность работы веб-интерфейса при смене поискового запроса, подтверждая, что ранее отмеченные фильтры не сбрасываются автоматически.

Результаты действий представлены на рисунках (см. рисунки 14 – 16).

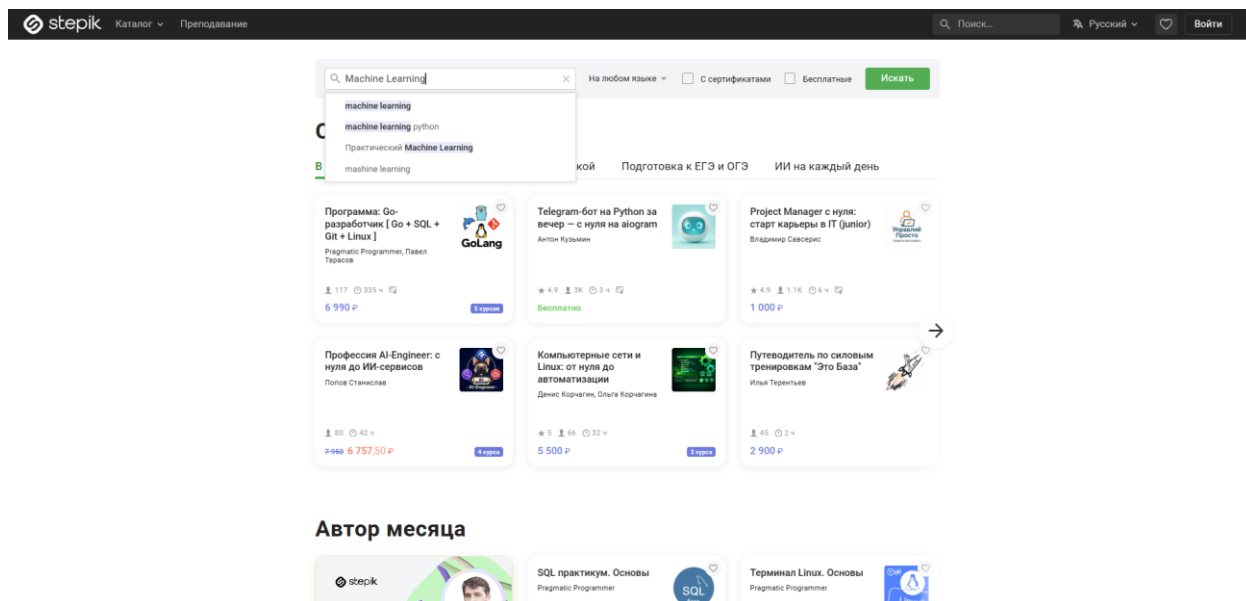


Рисунок 14 – Введение в поиск запрос «Machine Learning»

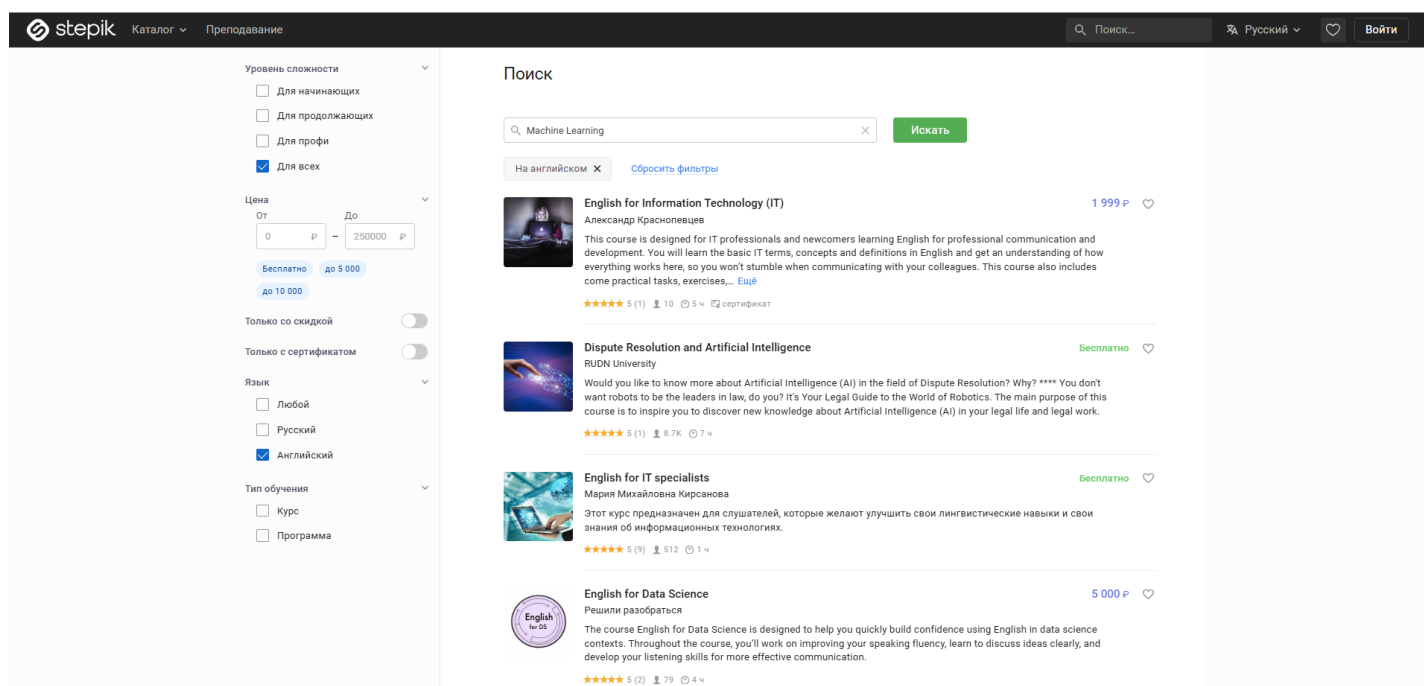


Рисунок 15 – Показ всех доступных курсов

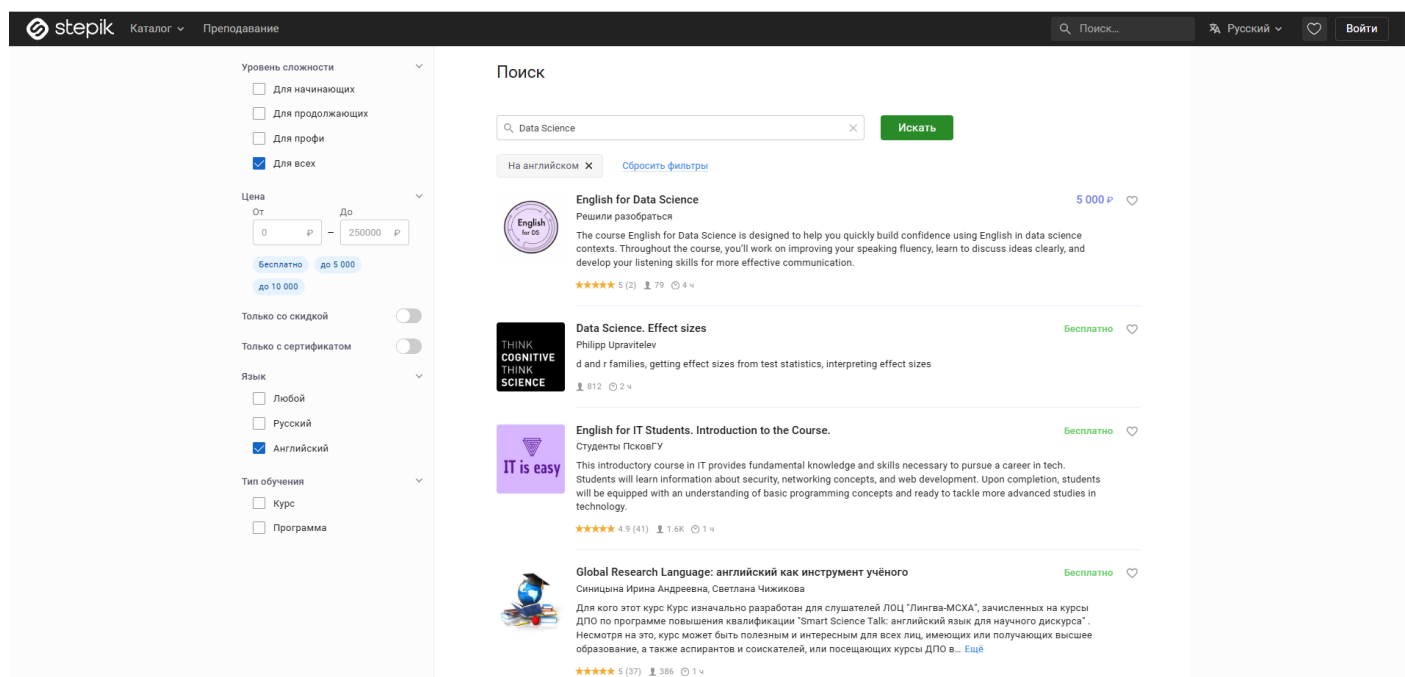


Рисунок 16 – Введение нового запроса «Data Science»

Тест №7: Несуществующий запрос с фильтрами

Тест контролирует обработку исключений при поиске, проверяя появление уведомления об отсутствии результатов при вводе некорректного запроса с активными фильтрами.

Результаты действий представлены на рисунках (см. рисунки 17, 18).

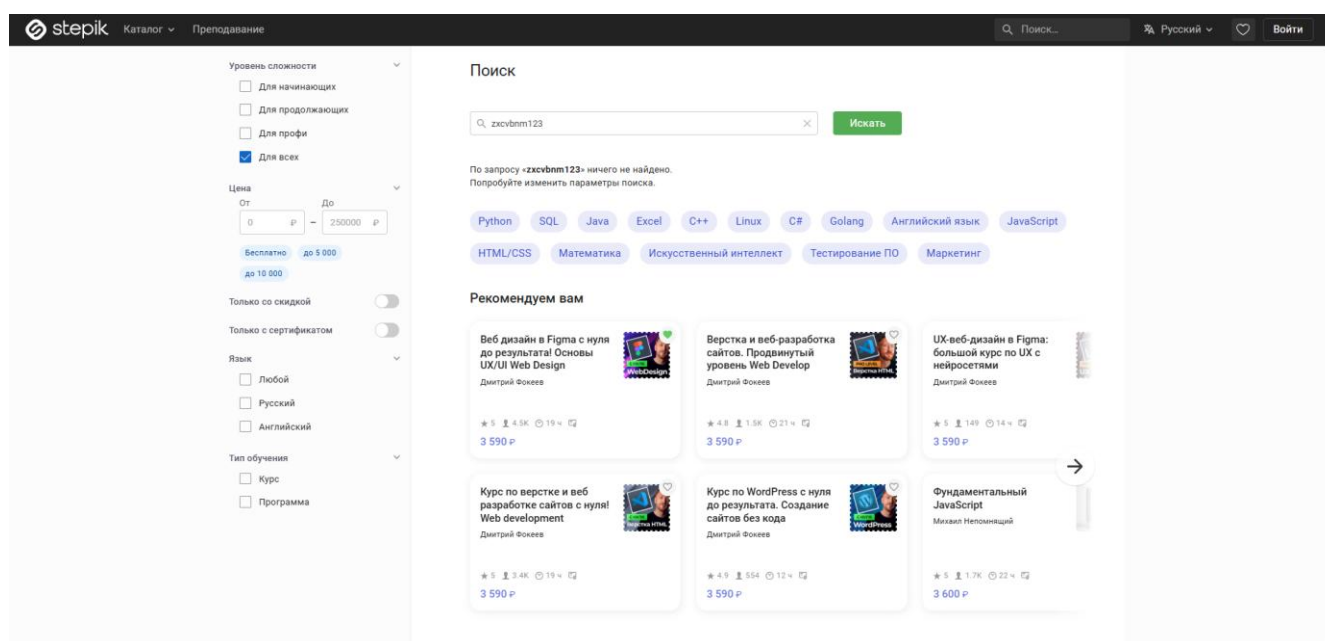


Рисунок 17 – Введение в поиск запрос «zxsvbnt123»

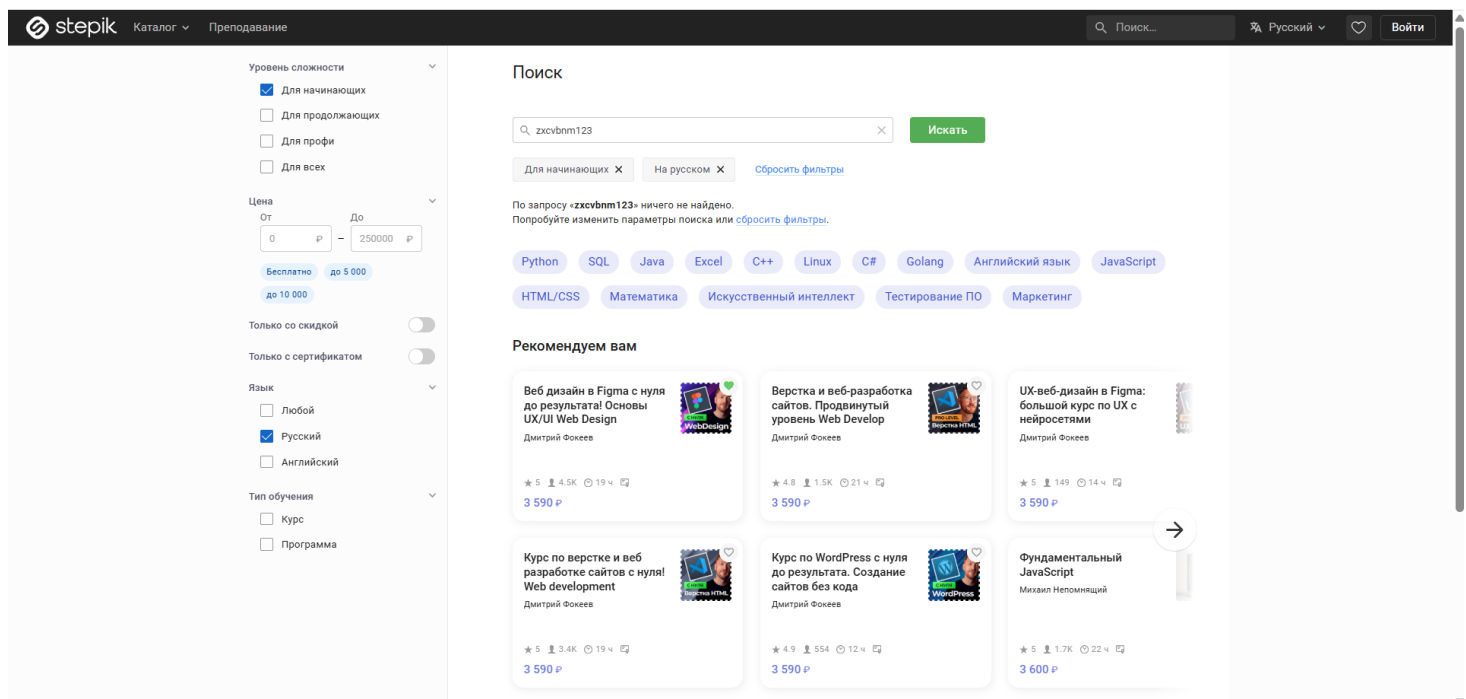


Рисунок 18 – Применение чекбоксов «Русский» и «Для начинающих»

Тест №8: Поиск по автору с переходом в профиль

Тест направлен на верификацию результатов поиска по имени конкретного автора и оценку корректности перехода по ссылке из карточки курса в его профиль для ознакомления с отзывами.

Результаты действий представлены на рисунках (см. рисунки 19 – 22).

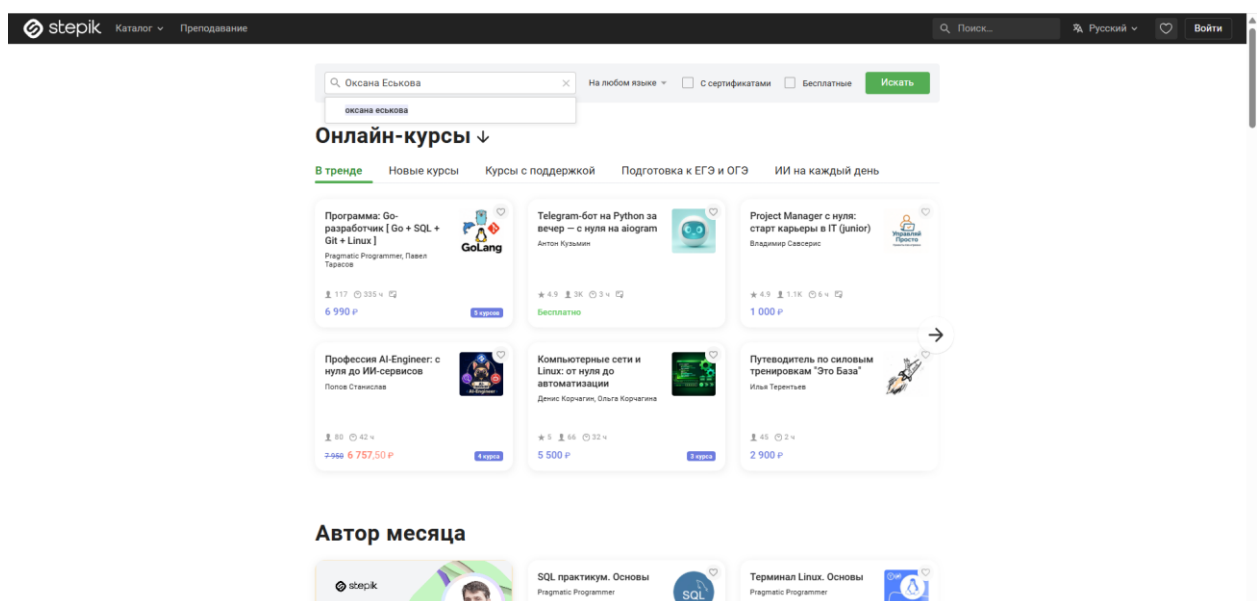


Рисунок 19 – Введение в поиск запрос «Оксана Еськова»

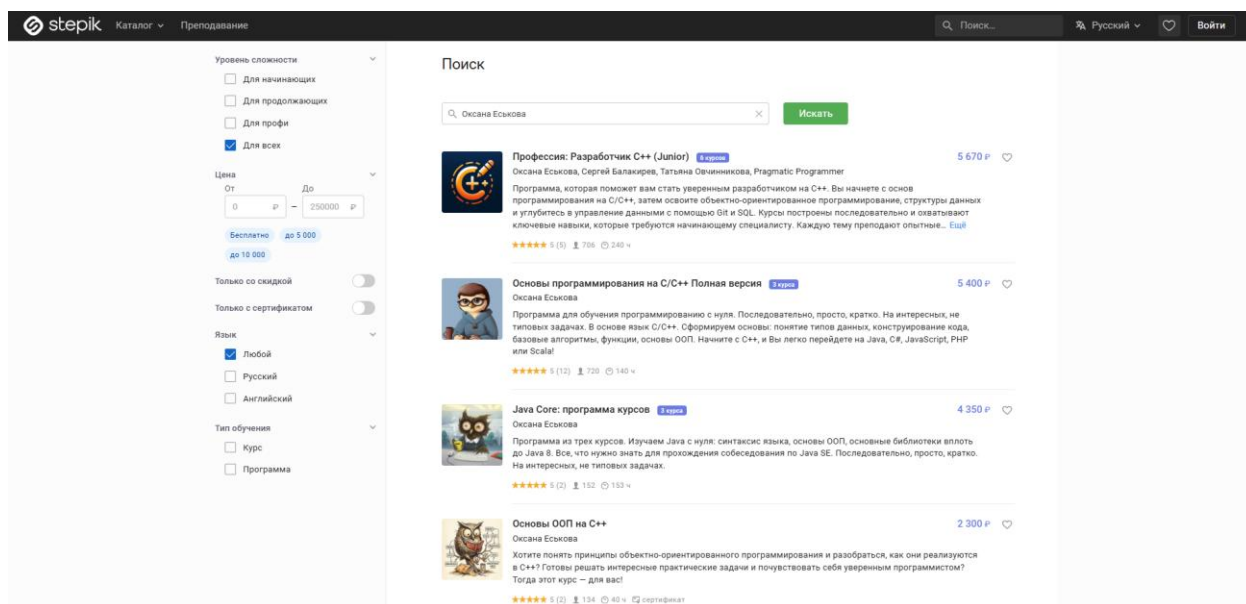


Рисунок 20 – Показ всех доступных курсов

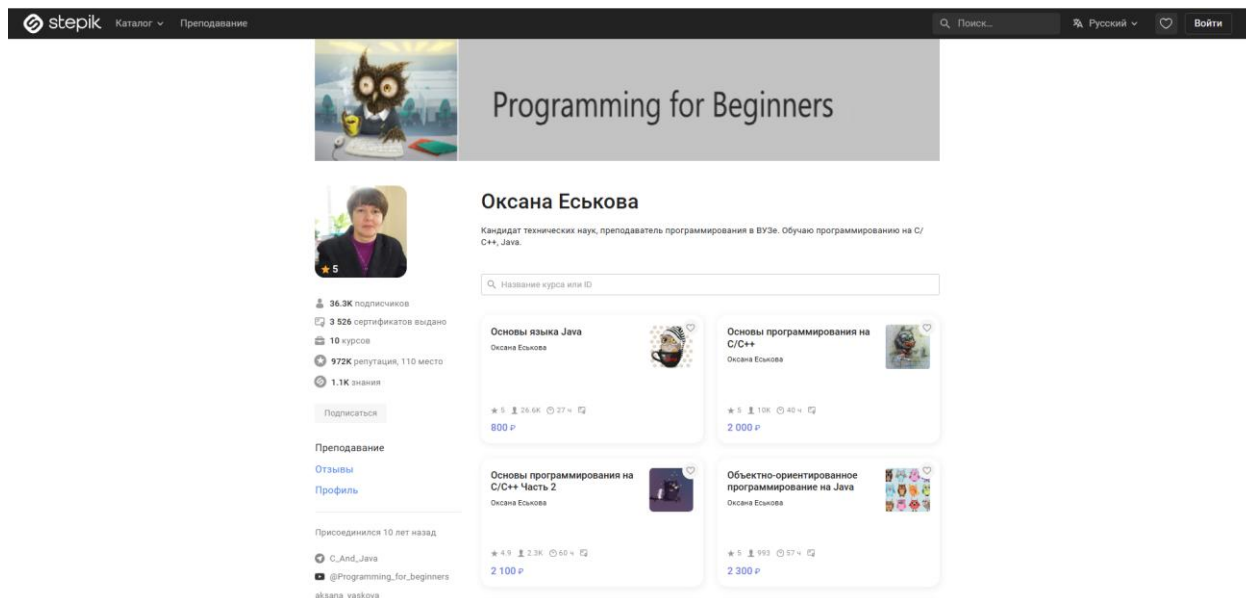


Рисунок 21 – Переход в профиль пользователя «Оксана Еськова»

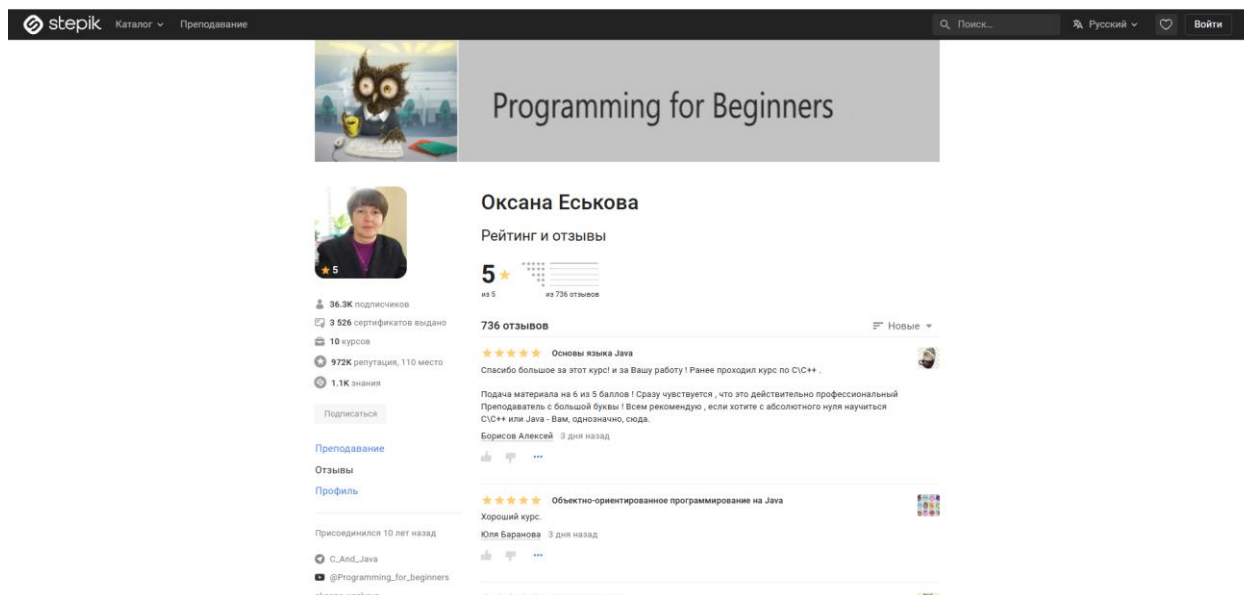


Рисунок 22 – Переход во вкладку «Отзывы»

Тест №9: Фильтрация акционных курсов

Данная проверка подтверждает работоспособность акционного фильтра-переключателя, проверяя отображение зачеркнутой стоимости в карточках отобранных курсов.

Результаты действий представлены на рисунках (см. рисунки 23, 24).

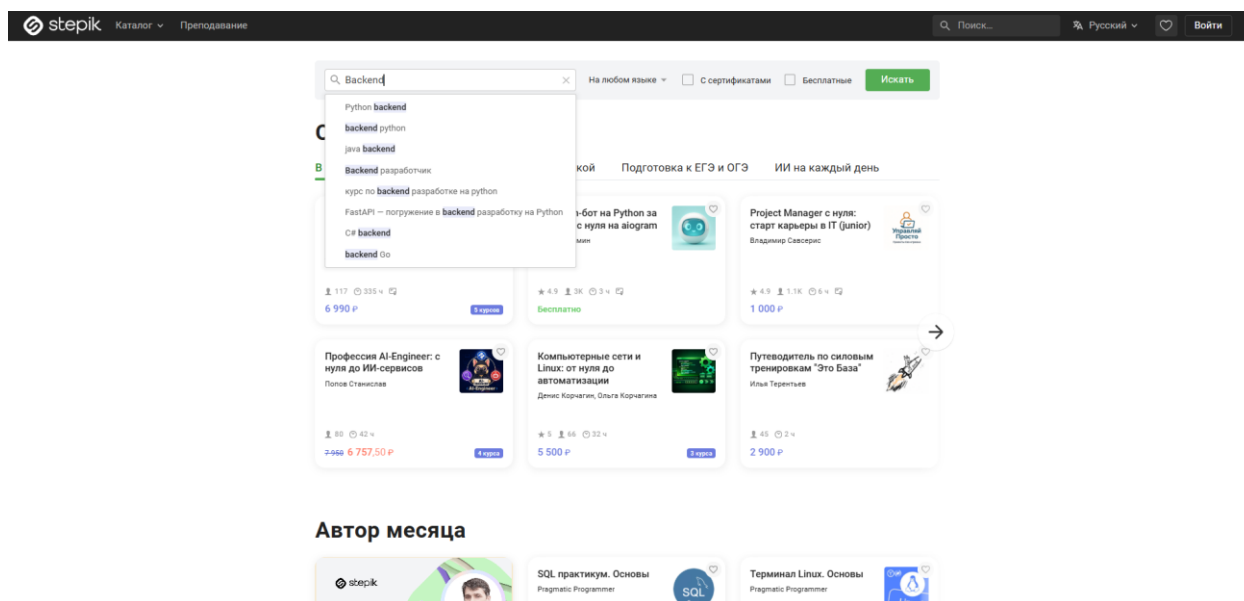


Рисунок 23 – Введение в поиск запрос «Backend»

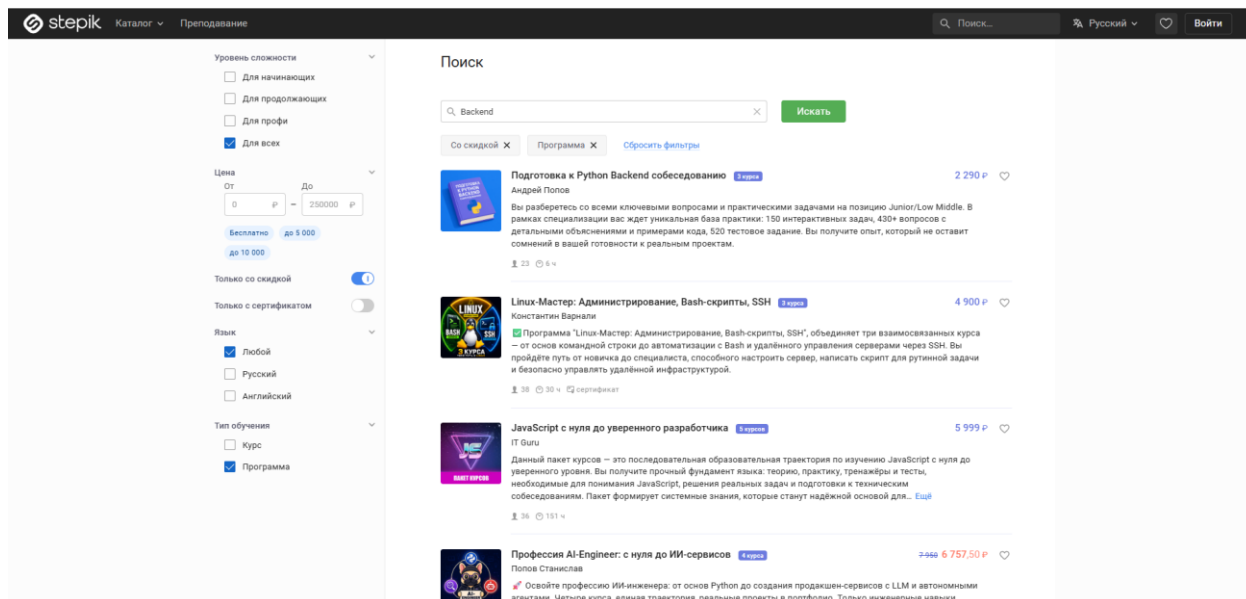


Рисунок 24 – Применение чекбоксов «Только со скидкой» и «Программа»

Тест №10: Добавление курса в избранное из поиска

Данная проверка контролирует процесс добавления курса в избранное из результатов поиска, подтверждая, что клик по иконке корректно переводит её в активное состояние.

Результаты действий представлены на рисунках (см. рисунки 25 – 27).

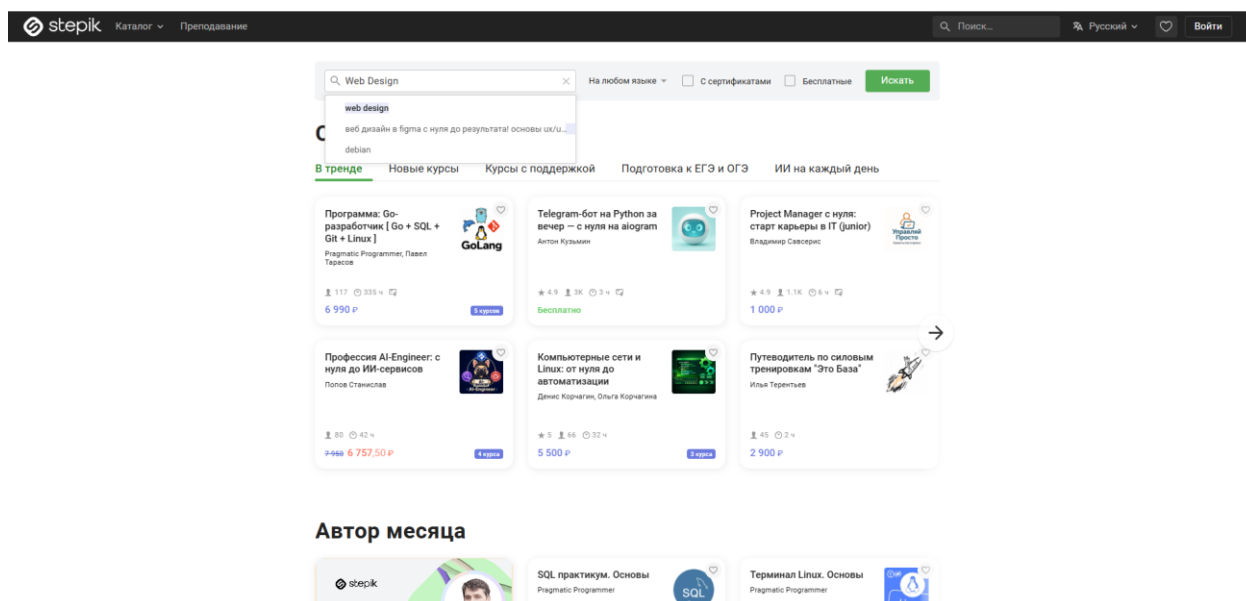


Рисунок 25 – Введение в поиск запрос «Web Design»

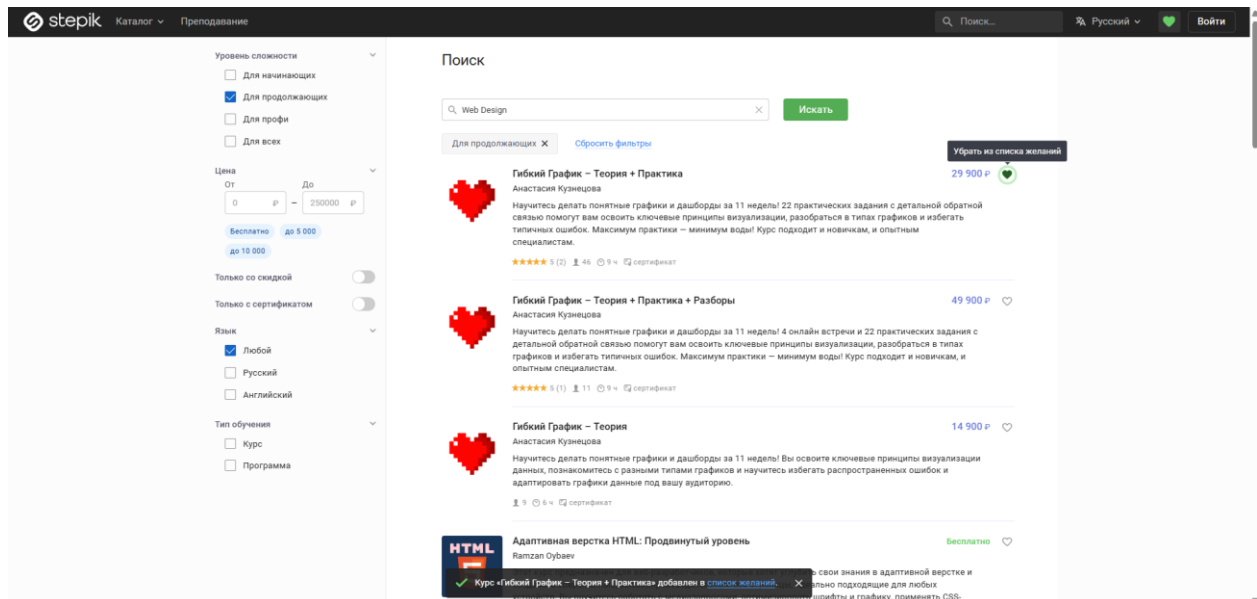


Рисунок 26 – Добавление первого курса в избранное

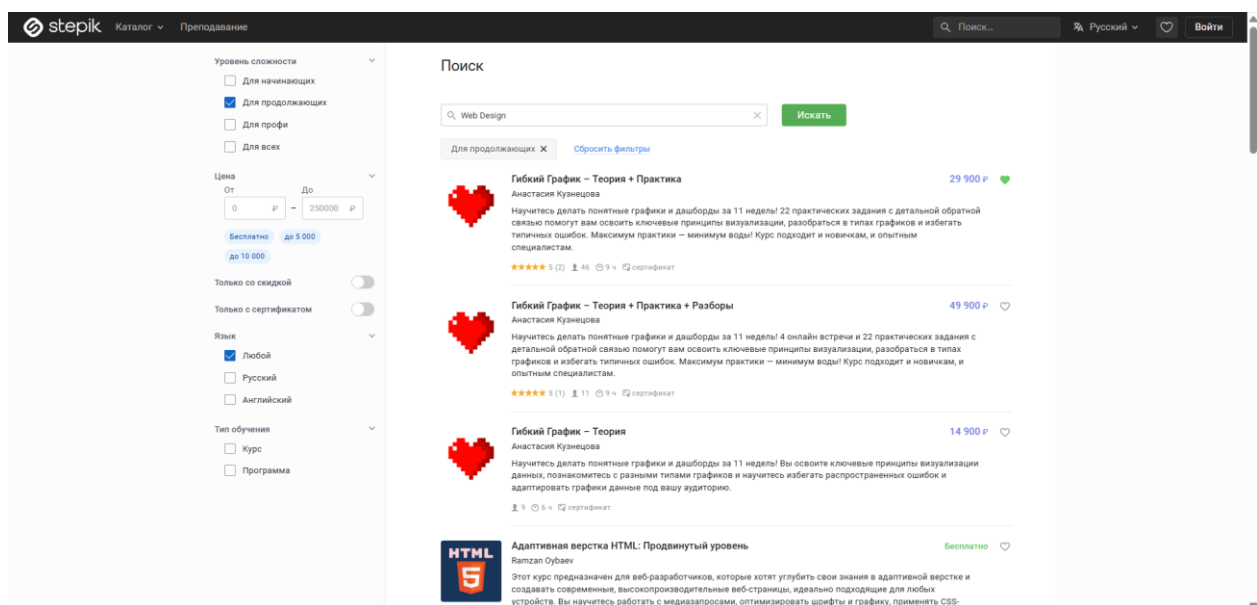


Рисунок 27 – Показ того, что первый курс в избранном (сердечко стало закрашенным)

Тесты поделены на два блока: поиск и фильтрация.

Результаты выполнения первой группы тестов представлены на рис. 28:

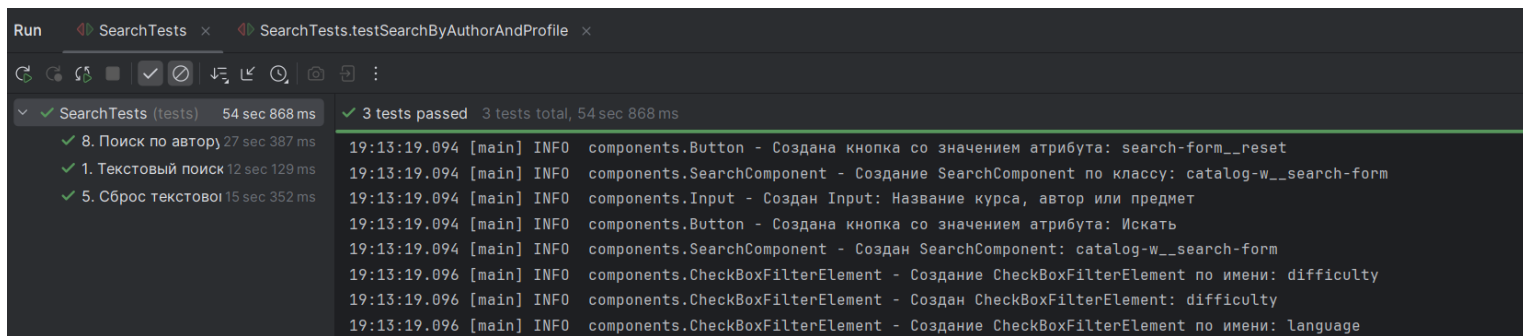


Рисунок 28 – Результаты первого блока

Результаты выполнения второй группы тестов представлены на рис. 29

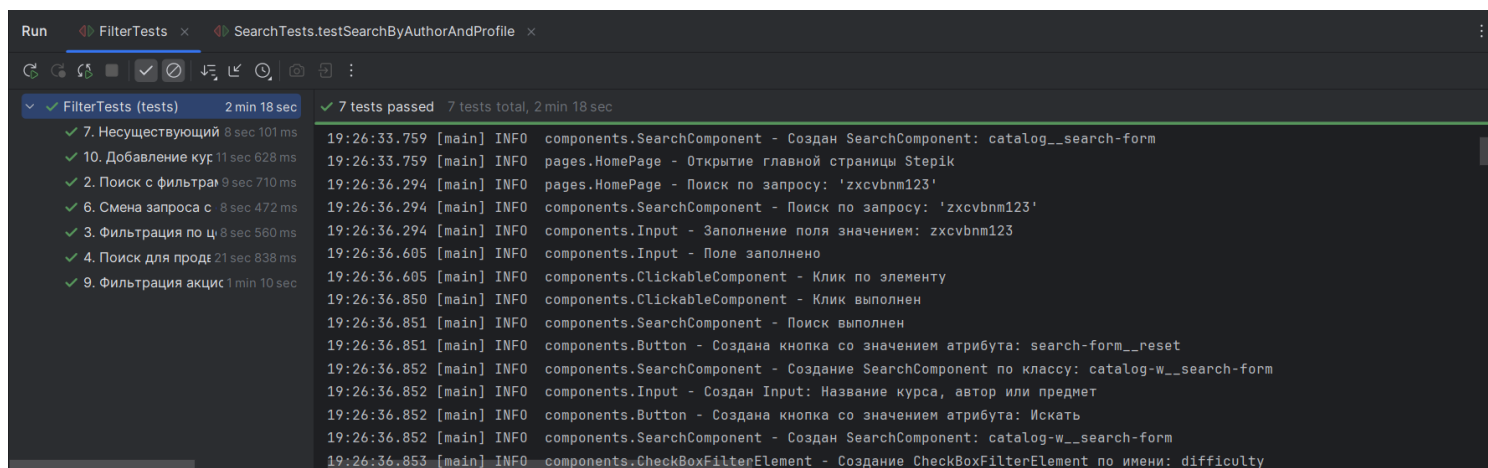


Рисунок 29 – Результаты второго блока

2 РЕЗУЛЬТАТЫ РАБОТЫ

В данном разделе приведено описание архитектуры разработанной системы автоматизированного тестирования. В основу проекта заложен паттерн проектирования Page Object Model (POM), который позволяет разделить логику тестов и описание элементов пользовательского интерфейса, обеспечивая высокую расширяемость и удобство поддержки кода.

2.1. Описание программных классов и их методов

Программный комплекс разделен на несколько логических пакетов: компоненты, страницы, тесты и вспомогательные утилиты.

Пакет **components** (UI элементы):

1. **BaseComponent**: Абстрактный базовый класс, выступающий в роли объектно-ориентированной обертки над базовым интерфейсом **SelenideElement**. Инкапсулирует общую логику поиска дочерних веб-элементов с помощью XPath-селекторов [5] и предоставляет методы безопасного доступа к ним.
2. **ClickableComponent**: Интерфейс, определяющий стандартное поведение для кликабельных элементов с обработкой ожидания видимости и логированием ошибок.
3. **Button**: Специализированный класс-компонент для взаимодействия с кнопками на веб-страницах. Реализует интерфейс **ClickableComponent**. Содержит методы создания кнопки по названию (**byName**) или по CSS-классу (**byClass**).
4. **Input**: Специализированный класс, инкапсулирующий логику работы с текстовыми полями ввода. Содержит методы инициализации поля по атрибуту **placeholder** (**byPlaceHolder**), заполнения поля (**fill**) и получения текущего строкового значения (**getValue**).
5. **CheckBox**: Специализированный класс для взаимодействия с чекбоксами. Реализует методы интерфейса **ClickableComponent** и предоставляет метод инициализации чекбокса по атрибуту **data-qa-value** (**byValue**).

6. `CheckBoxFilterElement`: Класс для работы с группой чекбоксов внутри блока фильтрации. Содержит методы создания фильтра по имени (`byName`) и получения чекбокса по значению внутри группы (`getCheckBoxByValue`).
7. `SearchComponent`: Составной компонент, объединяющий строку поиска и кнопку отправки запроса. Содержит методы инициализации по CSS-классу (`byClass`), выполнения поиска (`search`) и получения текущего значения поля (`getCurrentInputValue`).
8. `FilterComponent`: Составной компонент, инкапсулирующий логику работы со всеми фильтрами на странице поиска: фильтрация по цене, сложности, языку и типу курса. Содержит методы применения чекбоксов (`filterByCheckBox`), ценового фильтра (`filterByMinPrice`, `filterByMaxPrice`), тумблеров (`filterByToggle`) и проверки состояния чекбокса (`isCheckBoxSelected`).
9. `PriceFilterElement`: Компонент для работы с ценовым фильтром. Содержит поля ввода минимальной и максимальной цены. Содержит методы получения полей ввода (`getminValueInput`, `getmaxValueInput`).
10. `ToggleFilterElement`: Класс для работы с тумблерами-переключателями. Содержит методы создания по имени (`byName`), переключения (`toggle`) и проверки активности (`isActive`).
11. `CourseCardItem`: Класс для работы с отдельной карточкой курса в поисковой выдаче. Реализует методы добавления в избранное (`addToWishlist`), клика по первому автору (`clickFirstAuthor`), получения заголовка (`getTitle`), проверки старой цены (`hasOldPrice`) и проверки наличия в избранном (`isFavorite`).
12. `CourseCardText`: Класс для работы с текстовым элементом внутри карточки курса (название или автор). Реализует интерфейс `ClickableComponent`. Содержит метод получения текста (`getText`).

Пакет `pages` (Page Objects):

1. `BasePage`: Абстрактная основа для всех страниц, содержащая общие параметры ожидания и методы инициализации.
2. `HomePage`: Описывает главную страницу каталога и иницирует процесс поиска.
3. `SearchResultsPage`: Центральный класс, управляющий результатами поиска, применением фильтров и навигацией по страницам (пагинацией).
4. `CoursePage` и `ProfilePage`: Классы для взаимодействия со страницей конкретного курса и профилем автора соответственно.

Пакет `helpers` и `tests`:

1. `WindowHandler`: Вспомогательный класс для управления окнами и вкладками браузера.
2. `BaseTest`: Класс конфигурации, отвечающий за настройку драйвера перед запуском тестов и закрытие сессии после их завершения.
3. `SearchTests`: Основной тестовый класс, содержащий сценарии текстового поиска, проверки пустых запросов, поиска со спецсимволами и навигации по результатам выдачи.
4. `FilterTests`: Тестовый класс, объединяющий сценарии проверки различных фильтров (цена, язык, сложность, акции). Вынесение этих тестов в отдельный класс позволило логически разделить функционал поиска и фильтрации.
5. `TestConstants`: Класс-хранилище статических констант. В него вынесены все «магические числа» (таймауты, ожидаемые тексты, тестовые наборы данных), что обеспечивает удобство редактирования тестов без изменения программной логики.

2.2. UML-диаграмма классов

На рисунке (см. рис. 30) приведена UML-диаграмма, отображающая структуру классов, их иерархию наследования, реализацию интерфейсов и взаимосвязи между компонентами и страницами.

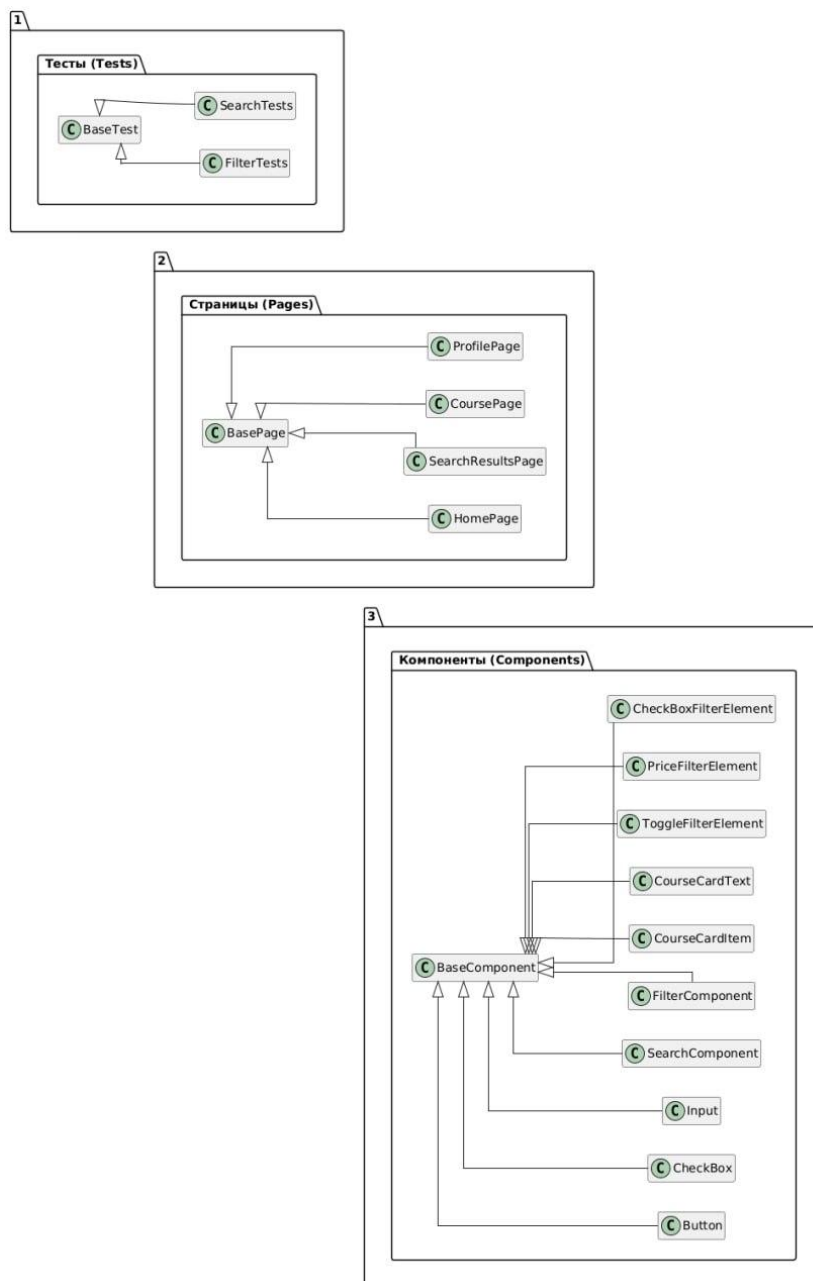


Рисунок 30 - UML-диаграмма

2.3. Реализация взаимодействия компонентов

Связь между классами построена по иерархическому принципу. Тестовые методы никогда не работают с элементами сайта напрямую. Вместо этого они вызывают готовые методы классов страниц (Page Objects). Страницы же, если нужно выполнить действие со специфическими блоками интерфейса, передают эту задачу составным компонентам (Components).

Например, если нужно ограничить стоимость курса и происходит вызов метода `applyMinPriceFilter()` в классе `SearchResultsPage` происходит обращение к вложенному объекту `FilterComponent`, который взаимодействует с объектом `Input`. Такая схема очень удобна в работе. Если на сайте Stepik изменится кнопка или поле ввода, необходимо будет обновить к ней путь только один раз. Сами тесты при этом останутся рабочими и менять в них ничего не придется.

3 ИНДИВИДУАЛЬНАЯ РАБОТА

В рамках работы над проектом автоматизации тестирования образовательной платформы Stepik была выполнена часть задач, связанная с проектированием, реализацией и отладкой системы тестового фреймворка на основе паттерна Page Object Model (POM).

Конкретные задачи, выполненные лично:

1. Программная реализация тестовых сценариев №1-5

Были разработаны, отлажены и подготовлены к запуску первые пять автоматизированных сценариев тестирования:

- Тест 1 (Текстовый поиск и навигация по страницам) - проверка ввода текстового запроса, перехода на вторую страницу выдачи и проверка заголовка открытого курса в новой вкладке браузера.
- Тест 2 (Поиск с фильтрами) - проверка совместной работы текстового поиска, чекбокса языка интерфейса и типа программы обучения.
- Тест 3 (Фильтрация по ценовому диапазону) - проверка корректности ввода минимальной и максимальной стоимостей курса и фильтрации выдачи.
- Тест 4 (Поиск для продвинутых с сертификатом) - проверка фильтрации по повышенной сложности и переключателя наличия сертификата.
- Тест 5 (Сброс текстового поиска «крестиком») - проверка очистки поискового ввода на второй странице результатов и корректного возврата каталога в исходное состояние.

2. Повышение стабильности выполнения автотестов

Проведена детальная отладка разработанных сценариев для устранения факторов нестабильности:

- Внедрены явные ожидания Selenide на смену URL-адреса и появление элементов выдачи, что устранило проблемы преждевременных ассертов до полной загрузки AJAX-запросов браузером.

3. Доработка и расширение методов объектных моделей страниц

Для обеспечения работоспособности тестов, были изменены и добавлены новые методы в классах страниц и компонентов:

- В класс `SearchResultsPage.java` были добавлены два новых метода `applyMinPriceFilter` и `applyMaxPriceFilter` вместо одного общего метода установки диапазона. Это позволило разделить заполнение полей «Цена От» и «Цена До» на два независимых и последовательных тестовых действия в соответствии с обновленными шагами чек-листа.
- В классе `FilterComponent.java` были реализованы методы `filterByMinPrice` и `filterByMaxPrice`. Они принимают отдельные строковые значения стоимости и делегируют управление вводом соответствующих стоимостных границ дочерним полям ввода ценового фильтра.
- В классе `PriceFilterElement.java` была скорректирована структура работы с ценовым диапазоном и логика работы методов прямого доступа к полям ввода минимальной и максимальной цены (`getminValueInput` и `getmaxValueInput`), что обеспечило возможность их отдельного независимого заполнения.

4. Рефакторинг и приведение кодой базы тестов к единому стандарту

Проведен рефакторинг тестовых классов для улучшения поддерживаемости и чистоты кодовой базы:

- Тесты, выполняющие проверку многокритериальной фильтрации, были логически отделены от базовых тестов поиска и навигации и вынесены в отдельный класс `FilterTests.java`.
- Из кода тестов были удалены временные комментарии с описанием действий. Вместо них было настроено логирование происходящих действий в консоль, что сделало код чистым, а логи более информативными.

4 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ

В рамках учебной практики был подобран технологический стек, ориентированный на надежность функционирования и простоту поддержки разработанного проекта. Основным языком программирования выступил Java. Это помогло применить принципы объектно-ориентированного проектирования и выстроить понятную архитектуру с отдельными уровнями ответственности для страниц, компонентов и тестов. Для сборки проекта использовался фреймворк Maven.

Автоматизацию работы с веб-интерфейсом платформы Stepik удалось реализовать с помощью фреймворка Selenide. Данный выбор обусловлен его возможностями при работе с динамическими элементами, а также встроенными «умными» ожиданиями, что повышает устойчивость тестов. Запуск сценариев координирует библиотека JUnit 5, а за протоколирование хода выполнения и локализацию ошибок отвечает интегрированная система логирования Log4j2.

Подобный стек технологий помог сформировать масштабируемую структуру автотестов и автоматизировать базовые проверки поиска и фильтрации в каталоге Stepik.

5 ОТЗЫВ РУКОВОДИТЕЛЯ

По результатам работы учебная практика оценена на <ваша_оценка>.

ЗАКЛЮЧЕНИЕ

В ходе выполнения учебной практики были успешно достигнуты все поставленные цели и решены задачи по разработке и отладке системы автоматизированного UI-тестирования для образовательной платформы Stepik.org.

На первом этапе был проведен анализ функциональных возможностей веб-приложения, что позволило выявить критические сценарии взаимодействия пользователя с поисковой строкой и фильтрами. Результатом этой работы стала детальная программная реализация первых пяти тестовых сценариев из чек-листа: текстового поиска и навигации по страницам поиска с фильтрами языка интерфейса и типа обучения, отдельной фильтрации по ценовому диапазону, поиска для продвинутых с сертификатом и сброса текстового поиска с помощью интерактивного «крестика».

Задача по реализации архитектуры проекта на языке Java была успешно решена с использованием паттерна Page Object Model и компонентно-ориентированного подхода. Была проведена доработка и рефакторинг классов страниц и компонентов для разделения ввода стоимостных границ. В классы `SearchResultsPage.java` и `FilterComponent.java` были внедрены отдельные методы `applyMinPriceFilter`, `applyMaxPriceFilter`, `filterByMinPrice` и `filterByMaxPrice` вместо одного общего метода установки диапазона. Также была скорректирована структура работы с ценовым диапазоном в классе `PriceFilterElement.java` и логика работы методов прямого доступа к полям ввода минимальной и максимальной цены (`getminValueInput` и `getmaxValueInput`), что обеспечило возможность их отдельного независимого заполнения в соответствии с шагами чек-листа.

Программная реализация тестов и логирования была успешно выполнена с использованием современных библиотек Selenide и JUnit 5, а также системы логирования Log4j2. Для повышения стабильности выполнения автотестов и устранения факторов нестабильности были внедрены явные ожидания Selenide на смену URL-адреса и появление

элементов выдачи, что полностью устранило проблемы преждевременных ассертов до полной загрузки AJAX-запросов браузером. Был проведен масштабный рефакторинг тестового набора: все сценарии, проверяющие многокритериальную фильтрацию, были логически отделены от базового поиска и вынесены в обособленный класс `FilterTests.java`. Из кода автотестов были удалены временные комментарии с шагами, а логирование хода выполнения тестов приведено к лаконичному и единому стилю.

В целом, прохождение практики позволило закрепить практические навыки объектно-ориентированного проектирования и освоить современные инструменты промышленной автоматизации тестирования. Все разработанные программные классы приведены к единому стандарту кодирования, снабжены документацией `JavaDoc` и размещены в удаленном репозитории `GitHub` [6], что гарантирует сохранность наработок, возможность доработок и совместный доступ к результатам тестирования.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Fowler, M. PageObject [Электронный ресурс].
URL: <https://martinfowler.com/bliki/PageObject.html> (дата обращения: 07.07.2026).
2. Selenide: Concise UI Tests in Java [Электронный ресурс].
URL: <https://selenide.org/> (дата обращения: 08.07.2026).
3. JUnit 5 User Guide [Электронный ресурс].
URL: <https://junit.org/junit5/docs/current/user-guide/> (дата обращения: 07.07.2026).
4. Apache Log4j2 Manual [Электронный ресурс]. URL: <https://logging.apache.org/log4j/2.x/manual/index.html> (дата обращения: 10.07.2024).
5. Язык запросов XPath — руководство MSiter [Электронный ресурс].
URL: <https://msiter.ru/tutorials/xpath> (дата обращения: 07.07.2026).
6. <https://github.com/vyixwq/Stepik-practice>